# Sir Syed University of Engineering & Technology
## END SEMESTER EXAMINATIONS FALL 2022
**SOLUTION**

| | | | | | | |
|---|---|---|---|---|---|---|
| **Faculty:** | Computing & Applied Sciences | | **Department:** | Computer Science & Information Technology | **Batch:** | 2021F |
| **Semester:** | 3rd | **Degree Program:** | BS (Computer Science) | **Course Instructor(s):** | Mr. Shakir Karim / Dr. Abdul Hameed Pitafi | |
| **Course Code:** | CS-215T | **Course Title:** | Computer Organization & Assembly Language | | **Credit Hours:** | 3 |
| **Date of Examination:** | 01-02-2023 | **Timings:** | 9:30 am to 12:30 pm | **Shift:** | Morning | |

Timings: 3 Hours Max Marks: 50

**Instructions:**
- Please don't write anything on the Question Paper except for your Roll# and Name.
- The paper must be solved / attempted on Printed Answer books provided by the Examinations Department.
- Please return the Question Paper along with the Answer books.

| **Roll #** | | **Name** | **SAMPLE SOLUTION** | **Section** | | **'A' Copy Serial No.** | |
|---|---|---|---|---|---|---|---|

| **Q.No.1** | **[6+4]** |
|---|---|

**(a)** Differentiate between RISC and CISC processors on the basis of execution time. Also highlight the main characteristics of a RISC processor.

**Answer (a)**

**Execution Time = Time/Program = (Time/Cycle)x(Cycle/Instruction)x(Instruction/Program)**

RISC stands for Reduced Instruction Set Computers whereas CISC stands for Complex Instruction Set Computers. RISC is focused on the less cycle per instructions whereas CISC is focused on less storage. RISC consists of minimal instructions whereas CISC consists of large number of instructions.

**Characteristics of RISC:**
- One instruction per cycle
- Register-to-register operations
- Simple addressing modes

Simple instruction formats

**(b)** Consider the problem of swapping the bit pattern in a byte or a word as an application of the shift and rotate instruction. Write the Intel x86 Assembly Language based code that swaps the higher 8bits of a word to the lower 8bits. Assume that the word is stored in BX register and BX contains the value 64BAH.

**Answer (b)**

```
MOV BX, 64BAH
MOV CL, 8
ROR BX, CL
```

# Sir Syed University of Engineering & Technology
## END SEMESTER EXAMINATIONS FALL 2022
**SOLUTION**

| **Q.No.2** | **[5+5]** |
|---|---|

**(a)** Multiply **+13$_{10}$** by **–11$_{10}$** using Booth's algorithm. Show all the necessary steps involved.

**Answer (a):** M = **+13 = 01101** **Q = - 11 = 10101**

| Iteration # | $A_4A_3A_2A_1A_0$ | $Q_4Q_3Q_2Q_1Q_0$ | $Q_{-1}$ | $M_4M_3M_2M_1M_0$ | Remarks |
|---|---|---|---|---|---|
| Initializing | 0 0 0 0 0 | 1 0 1 0 1 | 0 | 0 1 1 0 1 | |
| Iteration #1 | 1 0 0 1 1<br>1 1 0 0 1 | 1 0 1 0 1<br>1 1 0 1 0 | 0<br>1 | 0 1 1 0 1 | A=A-M<br>SAR |
| Iteration #2 | 0 0 1 1 0<br>0 0 0 1 1 | 1 1 0 1 0<br>0 1 1 0 1 | 1<br>0 | 0 1 1 0 1 | A=A+M<br>SAR |
| Iteration #3 | 1 0 1 1 0<br>1 1 0 1 1 | 0 1 1 0 1<br>0 0 1 1 0 | 0<br>1 | 0 1 1 0 1 | A=A-M<br>SAR |
| Iteration #4 | 0 1 0 0 0<br>0 0 1 0 0 | 0 0 1 1 0<br>0 0 0 1 1 | 1<br>0 | 0 1 1 0 1 | A=A+M<br>SAR |
| Iteration #5 | 1 0 1 1 1<br>**1 1 0 1 1** | 0 0 0 1 1<br>**1 0 0 0 1** | 0<br>1 | 0 1 1 0 1 | A=A-M<br>SAR |

**+13 x -11 = -143 = 1101110001$_2$ (2's complement of 143)**

**ALTERNATIVELY**

**Q = +13 = 01101** **M = - 11 = 10101**

| Iteration # | $A_4A_3A_2A_1A_0$ | $Q_4Q_3Q_2Q_1Q_0$ | $Q_{-1}$ | $M_4M_3M_2M_1M_0$ | Remarks |
|---|---|---|---|---|---|
| Initializing | 0 0 0 0 0 | 0 1 1 0 1 | 0 | 1 0 1 0 1 | |
| Iteration #1 | 0 1 0 1 1<br>0 0 1 0 1 | 0 1 1 0 1<br>1 0 1 1 0 | 0<br>1 | 1 0 1 0 1 | A=A-M<br>SAR |
| Iteration #2 | 1 1 0 1 0<br>1 1 1 0 1 | 1 0 1 1 0<br>0 1 0 1 1 | 1<br>0 | 1 0 1 0 1 | A=A+M<br>SAR |
| Iteration #3 | 0 1 0 0 0<br>0 0 1 0 0 | 0 1 0 1 1<br>0 0 1 0 1 | 0<br>1 | 1 0 1 0 1 | A=A-M<br>SAR |
| Iteration #4 | 0 0 0 1 0 | 0 0 0 1 0 | 1 | 1 0 1 0 1 | SAR |
| Iteration #5 | 1 0 1 1 1<br>**1 1 0 1 1** | 0 0 0 1 0<br>**1 0 0 0 1** | 1<br>0 | 1 0 1 0 1 | A=A+M<br>SAR |

**-11 x +13 = -143 = 1101110001$_2$ (2's complement of 143)**

# Sir Syed University of Engineering & Technology
## END SEMESTER EXAMINATIONS FALL 2022
**SOLUTION**

**(b)** Represent $\mathbf{2419_{10}}$ in 32-bit IEEE 754 floating-point format. Show all the necessary steps involved.

**Answer (b):**

Since single precision is 32 bit format.
(**01** Sign bit, **08** Biased Exponent bits, **23** Significand bits)
Here, Bias = $2^{8-1}-1 = 2^7-1 = 128-1 = 127$
$2419_{10} = 100101110011_2 = 1.00101110011 \text{x } 2^{11}$
Sign bit = **0** since number is +ve
Biased Exponent = True Exponent + Bias = 11 + 127 = $138_{10}$ = $\mathbf{10001010}_2$
Significand = $\mathbf{00101110011000000000000}_2$
$2419_{10}$ = $\mathbf{01000101000101110011000000000000}_2$ = **45173000h**

| **Q.No.3** | **[5+5]** |
|---|---|

Consider a memory system that uses 20-bits to address at the byte level, with a cache memory that uses a line size of 16-byte.

**(a)** Assume a direct mapped cache with a line field consists of 8 bits. Show the address format and determine the parameters: *Number of addressable units, Number of bytes in cache memory, Number of lines in cache, Tag size.*

**Answer (a):**

**Address Format for Cache Memory:**

< - - - - 8 bits Tag - - > + < - - - - 8 bits # of lines - - - > + < - - - 4 bits line size - - - >

**Address Format for Main Memory:**

< - - - - - - - - - - - - - 16 bits # of blocks - - - - - - - - - -> + < - - - 4 bits line size - - - >

Number of addressable units = $2^{20}$ = 1 MB
Number of bytes in cache memory = line size x # of lines = 16bytes x $2^8$lines = 4096 Bytes = 4 KB
Number of line in cache = $2^8$ = 256 lines
Tag size = $2^8$ = 256 Bytes

**(b)** Assume an associative cache. Show the address format and determine the parameters: *Number of blocks in main memory, Number of bytes in main memory, Block size, Tag size.*

**Answer (b):**

**Address Format for Cache Memory:**

< - - - - 16 bits Tag - - - - - - - - - - - - - - - - - - - - - - - - > + < - - - 4 bits line size - - - >

**Address Format for Main Memory:**

< - - - - - - - - - - - - - 16 bits # of blocks - - - - - - - - - - -> + < - - - 4 bits line size - - - >

Number of blocks in main memory = $2^{16}$ = 64 KB
Number of bytes in main memory = # of Blocks x Block size = 64 KB x 16 = 1 MB
Block size = Line size = 16 bytes (given)
Tag size = $2^{16}$ = 64 KB

# Sir Syed University of Engineering & Technology
## END SEMESTER EXAMINATIONS FALL 2022
**SOLUTION**

| **Q.No.4** | **[10]** |
|---|---|

Consider a non-pipelined processor with a clock rate of 2.5 gigahertz and average cycles per instruction of 4. The same processor is upgraded to a pipelined processor with five stages but due to the internal pipeline delay, the clock speed is reduced to 2 gigahertz. Assume there are no bubbles in the pipeline. Find the speed up achieved in this pipelined processor.

**Answer:**

Execution time in non-pipelined system = Number of clock cycles taken for 1 instruction
= 4 clock cycles = 4 x (1/frequency)
= 4 x (1/2.5 GHz) = 4 x 0.4ns = 1.6 ns

Execution time in pipelined system = Number of clock cycles taken for 1 instruction
= 1 clock cycles = 1 x (1/frequency)
= 4 x (1/2 GHz) = 1 x 0.5ns = 0.5 ns

Speed up = Execution time in non-pipelined system / Execution time in pipelined system
= 1.6 ns / 0.5 ns = 3.2

# Sir Syed University of Engineering & Technology
## END SEMESTER EXAMINATIONS FALL 2022
**SOLUTION**

| **Q.No.5** | **[10]** |
|---|---|

Refer to the table I and instruction set summary (for *MOV,* and *ADD* commands), write machine code for the given instructions.

(i) **ADD AL, [0200]**  (iv) **ADD BL, CH**
(ii) **MOV CL, [BX]**  (v) **MOV SS, 1234H**
(iii) **ADD DX, [BP]**

**Answer:**

**(i) ADD Register <= Memory (000000dw mod reg r/m)**
d=1 since register is destination
w=0 since register is 8 bits wide
mod=00 since memory without displacement is used
reg=000 since AL is used as register
r/m=110 since DIRECT address is used
**00000010 00 000 110 0002 = 0206 0002h**

**(ii) MOV Register <= Memory (100010dw mod reg r/m)**
d=1 since register is destination
w=0 since register is 8 bits wide
mod=00 since memory without displacement is used
reg=001 since CL is used as register
r/m=111 since [BX] is used
**10001010 00 001 111 = 8A0Fh**

**(iii) ADD Register <= Memory (000000dw mod reg r/m)**
d=1 since register is destination
w=1 since register is 16 bits wide
mod=01 since memory with 8 displacement is used (with displacement of 0 bytes)
reg=010 since DX is used as register
r/m=110 since [BP]+8 address is used
**00000011 01 010 110 00000000 = 0356 00h**

**(iv) ADD Register <= Register (000000dw mod reg r/m)**
d=0 since register is source
w=0 since register is 8 bits wide
mod=11 since Register addressing is used
reg=101 since CH is used as register
r/m=011 since BL is used
**00000000 11 101 011 = 00EBh**

**(v) MOV Segment Register <= Immediate**
**Illegal Instruction**

# Sir Syed University of Engineering & Technology
## END SEMESTER EXAMINATIONS FALL 2022

**SOLUTION**

**Table I: R/M and MOD values**

| R/M \ MOD | 00<br>0-bit displacement | 01<br>8-bit displacement | 10<br>16-bit displacement | 11<br>w=0 | 11<br>w=1 |
|---|---|---|---|---|---|
| **000** | [BX+SI] | [BX+SI]+D8 | [BX+SI]+D16 | AL | AX |
| **001** | [BX+DI] | [BX+DI]+D8 | [BX+DI]+D16 | CL | CX |
| **010** | [BP+SI] | [BP+SI]+D8 | [BP+SI]+D16 | DL | DX |
| **011** | [BP+DI] | [BP+DI]+D8 | [BP+DI]+D16 | BL | BX |
| **100** | [SI] | [SI]+D8 | [SI]+D16 | AH | SP |
| **101** | [DI] | [DI]+D8 | [DI]+D16 | CH | BP |
| **110** | DIRECT | [BP]+D8 | [BP]+D16 | DH | SI |
| **111** | [BX] | [BX]+D8 | [BX]+D16 | BH | DI |

**8088**

**8086/8088 Instruction Set Summary**

| Mnemonic and Description | Instruction Code | | | |
|---|---|---|---|---|
| **DATA TRANSFER** | | | | |
| **MOV = Move:** | 7 6 5 4 3 2 1 0 | 7 6 5 4 3 2 1 0 | 7 6 5 4 3 2 1 0 | 7 6 5 4 3 2 1 0 |
| Register/Memory to/from Register | 1 0 0 0 1 0 d w | mod reg r/m | | |
| Immediate to Register/Memory | 1 1 0 0 0 1 1 w | mod 0 0 0 r/m | data | data if w = 1 |
| Immediate to Register | 1 0 1 1 w reg | data | data if w = 1 | |
| Memory to Accumulator | 1 0 1 0 0 0 0 w | addr-low | addr-high | |
| Accumulator to Memory | 1 0 1 0 0 0 1 w | addr-low | addr-high | |
| Register/Memory to Segment Register | 1 0 0 0 1 1 1 0 | mod 0 reg r/m | | |
| Segment Register to Register/Memory | 1 0 0 0 1 1 0 0 | mod 0 reg r/m | | |
| **ARITHMETIC** | 7 6 5 4 3 2 1 0 | 7 6 5 4 3 2 1 0 | 7 6 5 4 3 2 1 0 | 7 6 5 4 3 2 1 0 |
| **ADD = Add:** | | | | |
| Reg./Memory with Register to Either | 0 0 0 0 0 0 d w | mod reg r/m | | |
| Immediate to Register/Memory | 1 0 0 0 0 0 s w | mod 0 0 0 r/m | data | data if s:w = 01 |
| Immediate to Accumulator | 0 0 0 0 0 1 0 w | data | data if w = 1 | |